



More trailing commas

Document number:

[P3776R0](#)

Date:

2025-08-27

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3776/github

Source:

github.com/eisenwave/cpp-proposals/blob/master/src/more-trailing-commas.cow



ABSTRACT: C++ should support trailing commas in more places, such as in function parameter lists.

§ Contents

- 1 Introduction**
 - 1.1 Recent history
 - 1.2 Trailing commas in other languages
- 2 Motivation**
 - 2.1 Improved text editing
 - 2.2 Improved version control
 - 2.3 Improved auto-formatter control
 - 2.4 Improved language consistency
 - 2.5 Eliminating some uses of `__VA_OPT__`
 - 2.6 Code generation convenience
 - 2.7 Motivating examples
- 3 Design**
 - 3.1 New trailing commas
 - 3.1.1 Addressing ambiguity concerns
 - 3.2 Already supported trailing commas
 - 3.3 Not proposed trailing commas
 - 3.3.1 Commas proposed in P0562R2
 - 3.3.2 Trailing commas in semicolon-terminated lists
 - 3.3.3 Non-lists
 - 3.3.4 Commas following ellipsis parameters
 - 3.3.5 Commas in macros
 - 3.4 Comparison to P0562R2
 - 3.5 Could this syntax be used for something else?
 - 3.6 Addressing criticisms
 - 3.6.1 Aesthetic objections

3.6.2	Concerns over C compatibility
3.6.3	Making previously ill-formed code valid
3.6.4	Semantic inconsistencies with macros
3.6.5	Multiple ways to do the same thing
4	Implementation experience
5	Impact on existing code
6	Wording
7	References



§ 1. Introduction

C++ permits the use of trailing commas in some lists, but not in others. For example, trailing commas are permitted at the end of an *initializer-list*, but not following a *mem-initializer-list*, *capture-list*, *template-parameter-list*, *expression-list*, in a *postfix-expression*, etc. It would be convenient if the support for trailing commas was extended to those other lists.

For example, the following should be valid:

```
void f(  
    int x,  
    int y, // ← trailing comma here  
);
```

§ 1.1. Recent history

While this proposal is new, a very similar proposal [\[P0562R2\]](#) has recently passed through EWG and CWG, but was ultimately not polled due to implementability concerns. [\[P0562R2\]](#) argues in favor of trailing commas following *mem-initializer-list* and *base-specifier-list*, with similar rationale as this proposal. The concern in question is that a comma after a *mem-initializer-list* introduces a parsing ambiguity between expressions and function bodies; see [\[P0562R2ReflectorDiscussion\]](#):

”

Classes are parsed in two phases: first, declarations are parsed, skipping the bodies of member functions and a few other things, and then those delayed parts are parsed in a context where the class is complete.

For most functions, skipping the body in the first pass is easy. There's an open brace, and you can just skip to the close brace. At worst, there's `try {`, and you need to find the end of the last catch handler.

The problem is that a *mem-initializer-id* can be a *template-id* that refers to a base class, and the identifiers it uses to name the base class might be declared (possibly after the constructor) in the same class — meaning that we've not parsed them yet. Therefore when we see a `<`, we don't know if it's introducing a template argument list. For example:

```
struct X {};  
struct Y : X {  
    Y() : A<b<c>(), {  
        // ...  
    }  
    // A, b, and c declared down here somewhere  
};
```

Are we in the body of the constructor yet?

Prior to P0562R2, the answer was no: an open brace preceded by a comma cannot start the constructor body. For this to be valid, `b` must be a template, and the `{` is then the start of the second template argument of `A`. ([...])

But after P0562R2, the above example seems to become ambiguous, and I'm not sure how an implementation would be able to parse it.

Following these developments in St. Louis 2024-06, the proposal has seen no activity. It seems like the baby has been thrown out with the bathwater here, since the rationale of the proposal is still sound and this ambiguity does not affect *base-specifier-lists*.



§ 1.2. Trailing commas in other languages

Various modern programming languages support trailing commas, not just in initialization of classes or when listing `enum` members, but also in function parameter lists or function argument lists:

Language	Trailing commas support
Rust	Has always supported trailing commas in. Use of trailing commas is recommended by the official style guide ([RustTrailingCommas]).
Kotlin	Added support for trailing commas in 2020 ([KotlinTrailingCommas]).
JavaScript	Support for trailing commas was standardized in ECMAScript2017 ([ECMAScriptTrailingCommas]).
TypeScript	Added support at the same time it was standardized for JavaScript ([TypeScriptTrailingCommas]).
C#	No support yet, in part due to concerns regarding tuple syntax, which also uses parentheses ([CSharpTrailingCommas]).
Swift	No support yet. Concerns regarding tuple syntax have also been raised ([SwiftTrailingCommas]).

The motivation for trailing commas in those languages equally applies to C++.



NOTE: More discussion on this language design choice can also be found at [[RedditTrailingCommas](#)] and [[OldNewThing](#)].

Various other languages such as Python, Go, and Julia also support trailing commas in function calls, but are not specially listed in the table above. That table aims to highlight particularly relevant examples with authoritative resources.

§ 2. Motivation

While trailing commas don't solve any major safety or performance issue, they improve developer convenience significantly in some ways. Given how many comma-separated lists developers regularly write, this convenience can be noticeable on a daily basis.



NOTE: The following motivation uses only function parameter lists as motivating examples, but the arguments equally apply to other comma-separated lists.

§ 2.1. Improved text editing

Advanced text editors typically have commands for cutting/copying whole lines, or let the developer reorder lines via keyboard shortcut. For example, a line can be swapped with the line above with `Alt + ↑` in VSCode. This can result in compiler errors without trailing commas:

```
void f(  
    int x,  
    int y  
);
```

When reordering `int x`, and `int y`, errors are raised:

```
void f(  
    int y // ↓ syntax error: "int" following "y"
```

```
int x, // ← syntax error: trailing comma
);
```



If both lines had a trailing comma and C++ permitted that syntax, reordering these lines would not be a problem.

§ 2.2. Improved version control

When an element is appended to a comma-separated list, this means that the previous element needs to receive a separating comma, resulting in menial changes:

```
void f(
    int x,
-   int y
+   int y,
+   int z
);
```

With trailing commas, we can turn a three-line change into a one-line change:

```
void f(
    int x,
    int y,
+   int z,
);
```

This smaller change is not just easier to review, it also does not pollute the revision history (`git blame`) for the previous line of code, and it does not introduce easily avoidable merge conflicts.

§ 2.3. Improved auto-formatter control

Many C++ developers auto-format their code using [ClangFormat](#). One powerful feature it has is the ability to control how brace-enclosed lists are formatted via the use of trailing commas.



EXAMPLE: ClangFormat may format C++ code like:

```
vector<int> numbers { LIST_ITEM_A, LIST_ITEM_B, //
                     LIST_ITEM_C };           // ← column limit here
                                              //
vector<int> numbers {
    LIST_ITEM_A,
    LIST_ITEM_B,
    LIST_ITEM_C, // ← trailing comma here
};
```

The compact version without a trailing comma may be situationally useful by compactifying code, allowing more to fit on screen. The verbose version with a trailing comma is more useful for regularly updated lists, and can be perceived as more organized. It's easy to envision the same example with function argument lists.

Trailing commas express the desire to format over multiple lines, which auto-formatters may utilize. Without support for trailing commas in e.g. function parameter lists, the developer is robbed of their ability to (elegantly) express that desire, since doing so would make the program ill-formed.



NOTE: A possible workaround for ClangFormat is to insert a trailing `//` at the end of a line (which forces line breaks to be retained), or to manually format a section of code with `// clang-format off`.

§ 2.4. Improved language consistency

It is generally surprising that C++ only supports trailing commas within a subset of its comma-separated lists. Besides the design not sparking joy, it creates practical problems.

For example, when refactoring code and e.g. converting list-initialization such as `{1, 2, 3,}` which already uses trailing commas into direct-initialization `(1, 2, 3,)`, the trailing comma introduces a compiler error until it is removed, which is a mild inconvenience at least.

§ 2.5. Eliminating some uses of `__VA_OPT__`

When we want to prepend an element to a variadic macro argument, we may need to use `__VA_OPT__` in the following way:

```
#define F(...) f(0 __VA_OPT__(,) __VA_ARGS__)
#define L(...) [& __VA_OPT__(,) __VA_ARGS__] {}
```

With an unconditional comma, `F()` would expand to `f(0,)`, which is currently ill-formed. A trailing comma in function calls and other lists would allow us to simplify such macros.



NOTE: While a *capture-default* like `&` is not part of the *capture-list*, a trailing comma should still be permitted following it. Otherwise, the example above would still be broken.

§ 2.6. Code generation convenience

Allowing trailing commas makes it easier to generate C++ code, for similar reasons as in §2.5. [Eliminating some uses of `\_\_VA\_OPT\_\_`](#). Generating lists is sometimes made easier if each element in a list can be terminated by a comma, rather than adding special handling for the first/last element to ensure no trailing commas in generated code.



EXAMPLE: Code generation for a list of function parameters is simplified by trailing commas in C++ as follows:

```
void emit_parameters(span<const Parameter> parameters) {
    emit('(');
    bool first = true;
    for (const auto& p : parameters) {
        if (!first) {
            emit(',');
        }
        first = false;
        emit_parameter(p);
        emit(',');
    }
    emit(')');
```

§ 2.7. Motivating examples

In discussions of this proposal, there is often some skepticism as to whether it's useful for specific lists. For example, I have been told that trailing commas may be useful in a *template-parameter-list*, but not in a *template-argument-list*. However, while some lists are more frequently multi-line, you can come up with a plausible multi-line example for virtually any list:

```
// template-parameter-list
template <
    input_iterator I,
    sentinel_for<I> S,
    weakly_incrementable O, // ← trailing comma here
>
copy_result function(I first, S last, O result);

// template-argument-list
using V = variant<
    int_least64_t,
    float,
```



```

double,
char8_t,
char16_t,
char32_t,
bool,
string, // ← trailing comma here
>;

// function-parameter-list
void pretty_print_log(
    log_level level,
    string_view message,
    optional<code_location> location = {},
    optional<const error&> error = {}, // ← trailing comma here
);

// function-argument-list
auto it = find(
    ranges::begin(container),
    ranges::end(container) + (container.size() / 2), // ← trailing comma here
);

// lambda-capture
auto callback = [
    &,
    current_index = 0,
    invocation_count = 0, // ← trailing comma here
] {
    ++invocation_count;
    // ...
};

// annotation-list
[[
    = testing::description("Fuzzing test for ASCII sequences."),
    = testing::execution::parallel, // ← trailing comma here
]]
void fuzz_ascii() { /* ... */ }

```

For any of these list types, it is plausible that they are long enough to be broken onto multiple lines, and it is plausible that changes will be made to such lists in the future, which makes a trailing comma useful in these lists.

§ 3. Design

I propose to add a trailing comma after a list whenever possible (i.e. no parsing issues are introduced), with the exception of lists terminated by a semicolon. The design can be summed up as:

” When enclosed by {}, (), [], or <>, lists can have a trailing comma.

All the following decisions are intended to make that rule of thumb correct. Cherry-picking individual cases like permitting `array[a, b, ]` but not `[a, b, ]{}` would only complicate the situation.

§ 3.1. New trailing commas

The following trailing commas are proposed, and not currently permitted.

<code>[: /* ... */ :]<A, B,></code>	<i>// template-argument-list in splice-specialization-specifier</i>
<code>[]<A, B,>{}</code>	<i>// template-parameter-list in lambda-expression</i>
<code>[a, b,]{}</code>	<i>// capture-list in lambda-introducer</i>
<code>d[a, b,]</code>	<i>// expression-list in subscript operator</i>
<code>f(a, b,)</code>	<i>// expression-list in call operator</i>



```

T(0,) // expression-list in function-style cast
typename T(0,) // expression-list in function-style cast with typename
new (a, b,) T // expression-list in new-placement
new T(a, b,) // expression-list in new-initializer
template for (int _ : { a, b, }) // expression-list in expansion-init-list
auto [a, b,] // sb-identifier-list in structured-binding-declaration
T f(a, b,) // parameter-declaration-list in parameter-declaration-clause
T x(a, b,); // expression-list in initializer
[ [=a, =b, ]] // annotation-list in attribute-specifier
S() : m(a, b,) // expression-list in mem-initializer
template<a, b,> // template-parameter-list in template-head
template<C<a, b,> T> // template-argument-list in type-constraint
template<template<A, B,> concept> // template-parameter-list in concept-tt-parameter
T<A, B,> // template-argument-list in simple-template-id
operator()<A, B,> // template-argument-list in template-id

```

While this initially appears like a huge change, it can be easily worded by incorporating the trailing comma directly into lists such as *expression-list*, since every occurrence of these lists would allow for trailing commas.

i NOTE: Of these, the *expression-list* in an *expansion-init-list* seems like a C++26 defect, and a CWG issue has been requested at [\[CWGGithub754\]](#). Since this appears to be a separate C++26 defect, not a new C++29 feature, this paper's wording does not include that fix.

i NOTE: Trailing commas would also be supported following a *capture-default*, such as in `[&, ]`. This is worth mentioning because a *capture-default* is not actually part of a *capture-list*, but a standalone element within a *lambda-capture*.

§ 3.1.1. Addressing ambiguity concerns

Note that all of these proposed cases are safe from parsing ambiguities encountered by [\[P0562R2\]](#). That is because a the trailing comma can only be followed by `'}', '}', ']',` or `'>'`. In the first three cases, a closing bracket cannot possibly be the beginning of a new element, only the end whatever encloses list. Neither `'>'`, `'>='`, nor `'>>'` are prefix unary operators, so in any case, `'>'` can only mean one thing.

Furthermore, `operator`, cannot be separated into `operator` and `'>'` or combined into `operator,>` (there is no `'>'` operator in the language), so no ambiguity can be introduced by having `operator` appear at the end of a list.

§ 3.2. Already supported trailing commas

The following trailing commas are already supported:

```

{ a, b, } // initializer-list in braced-init-list
{ .a=0, .b=0, } // designated-initializer-list in braced-init-list
enum { a, b, } // enumerator-list in enum-specifier
[[a,b,,,]] // attribute-list in attribute-specifier

```

§ 3.3. Not proposed trailing commas

§ 3.3.1. Commas proposed in P0562R2

The following two trailing commas are proposed by [\[P0562R2\]](#) and should be added, if at all possible, as a revision of that paper.

```

S() : a{ }, b{ }, // mem-initializer-list in ctor-initializer
struct S : A, B, { }; // base-specifier-list in base-clause

```

While the parsing ambiguity (§1.1. Recent history) seems limited to a *mem-initializer-list*, a *base-specifier-list* is similarly followed by `{`, which makes it quite plausible that similar parsing issues could affect that case. In any case, to keep this paper focused, I do not propose those two commas.

§ 3.3.2. Trailing commas in semicolon-terminated lists

Two more commas are not proposed because they are in semicolon-terminated lists:

```
int a, b;;           // init-declarator-list in simple-declaration
struct S { int a, b;; }; // member-declarator-list in member-declaration
friend A, B;;        // friend-type-specifier-list in friend-type-declaration
using a, b;;         // using-declarator-list in using-declaration
```

Firstly, for aesthetic reasons, it is unlikely that users would want to write this. Furthermore, multiple elements in a *init-declarator-list* are already discouraged by some style guides. Last but not least, the §2. Motivation of this paper focuses on multi-line scenarios. In the aforementioned cases, to make good use of trailing commas, developers would need to write:

```
int
    a,
    b,
;

using
    a,
    b,
;
```

Not only is this exotic, it is also more verbose than the status quo, which already allows reordering lines if we instead write:

```
int a;
int b;

using a;
using b;
```

Overall, permitting trailing commas in a *init-declarator-list* or *using-declarator-list* seems unmotivated, and would create unnecessary work for implementers.

§ 3.3.3. Non-lists

The following trailing commas are not proposed because they are not following a list:

```
static_assert(true, ",") // static_assert-declaration
contract_assert(true,)    // assertion-statement
= delete(" ",)           // deleted-function-body
```

While it may be more philosophically consistent to permit a comma within anything that vaguely looks like a function call, there is little motivation for such cases above. To be fair, a trailing comma could yield a minimal one-line change as follows:

```
static_assert(
    true,
+   "message"
);
```

However, it seems unlikely that a developer would choose to distribute a no-message `static_assert` over three lines in the first place. Overall, these cases just seem to add more work for implementers.

§ 3.3.4. Commas following ellipsis parameters

Another not proposed trailing comma is that after an ellipsis parameter:



```
void f(  
    int x,  
    ...,    // after ... in parameter-declaration-clause  
);
```

This decision is largely based on negative feedback in [\[CoreReflectorDiscussion\]](#).

The trailing comma in this place is largely unmotivated because the ellipsis is always the last element in the list. Consequently, a ... line cannot be reordered with the line before, and a comma will never have to be inserted after ... to accommodate a subsequent element.

§ 3.3.5. Commas in macros

The following trailing commas are not proposed:

```
#define M(x,y,) // identifier-list in control-line  
M(x,y,)        // arguments to a function-like macro
```

The former lacks motivation because it is highly unlikely that a C++ user would define multi-line parameters to a function-style macro. Doing so requires each parameter line to be terminated by \, among other reasons.

Supporting trailing commas in `M(x,y,)` would alter the meaning of existing code. Currently, that expression is providing three arguments to `M`, the last of which is empty.

§ 3.4. Comparison to P0562R2

It is worth noting that [\[P0562R2\]](#) did not propose trailing commas in function and template parameters with the following rationale:

“Function and template parameters do not allow a final terminating comma, but these do not trouble me in practice. (Arguably any function or template with enough parameters to be much of a maintenance issue seems like it is ripe for refactoring.) This paper does not propose changing function or template parameters.”

Note that [\[P0562R2\]](#) argues that additional trailing commas for *mem-initializer-lists* make “maintenance” easier. Maintenance equally motivates trailing commas in function parameter lists. However, [\[P0562R2\]](#) holds function parameter lists to a double standard where if you maintain such lists, they are “ripe for refactoring” anyway.

This position is hypocritical, lacks reason, and is out-of-touch with language development at large, where trailing commas in function parameter lists have recently been added to various programming languages (§1.2. [Trailing commas in other languages](#)). In fact, developers write tremendously more function parameter lists and argument lists than *mem-initializer-lists*, so the former are more deserving of attention.

§ 3.5. Could this syntax be used for something else?

The idea has been floated on some mailing lists to give empty list elements special meaning, such as using a default argument:

```
void f(int x = 0, int y = 0);  
  
f(, 1);    // OK, equivalent to f(0, 1)  
f(1, );    // OK, equivalent to f(1, 0)  
f(1,1,);   // error: third default argument requested, but only two parameters
```

Personally, I consider this idea terrible because of how non-descriptive it is; a user could even unintentionally perform such calls by making a typo ‘,’ somewhere in the argument list. Using the syntax `f(1,1,)` to mean anything other than a function call with two arguments and a trailing comma would be terribly confusing to developers accustomed to trailing commas from other languages, such as JavaScript or Kotlin developers (§1.2. [Trailing commas in other languages](#)).

§ 3.6. Addressing criticisms



§ 3.6.1. Aesthetic objections

I have unofficially polled the idea of trailing commas on social media prior to drafting this proposal, and a surprising amount of people objected to the idea. Note that this was a blind poll without explaining the purpose of the feature beforehand. When asking opponents of the idea why they don't like it, the answers revolved around aesthetics. In one case literally

” it looks ugly

Since trailing commas are foreign to both natural language and mathematical notation, I can sympathize with this position. However, we must keep two things in mind:

- Aesthetic perception is heavily influenced by familiarity. A Python developer likely perceives C++' use of special characters in `(!x && y)` or `int(T::*)` as "ugly". In practice, it is easy to get used to trailing commas. Since they are intended for multi-line scenarios, there is a fair amount of distance between ', ' and e.g. `>` anyway.
- The job of WG21 is to standardize a programming language used by software engineers. Trailing commas have clear technical benefits laid out in §2. Motivation, and clear technical benefits should always take precedence over matters of personal taste. If we had gotten hung up over `^^` and `[::]` "looking ugly", C++26 reflections would have died.

§ 3.6.2. Concerns over C compatibility

Some concerns have been raised regarding the fact that only C++ but not C would support the use of trailing commas. This was alleged to make writing common C and C++ code harder.

However, the use of trailing commas is entirely optional, so this claim is untrue. It is not a stated design goal of C++ to make every possible line of C++ code valid C as well. Instead, we maintain a common subset that is both valid C and valid C++. If a developer wants to write C/C++-interoperable code, they can simply not use trailing commas.

Furthermore, I intend to follow this proposal up with a WG14 counterpart if it finds consensus. For the sake of C++ compatibility, it is plausible that WG14 would adopt the syntax as well; this has been done with numerous features before.

§ 3.6.3. Making previously ill-formed code valid

Some people consider it a benefit that `f(0,)` is currently not allowed, which may catch mistakes in some scenarios:

” Maybe I was typing, distracted, and forgot my last argument? Maybe a macro expanded badly?

The problem with this line of reasoning is that there are infinite things that a developer may forget to type and infinite ways in which macros can expand into something unintended. Trailing commas are not particularly inviting to mistakes.

It is also important to keep one of C++'s language design rules in mind (see Design and Evolution of C++ — 4.3 Design Support Rules):

” It is more important to allow a useful feature than to prevent every misuse.

The usefulness of trailing commas clearly outweighs the benefits from this unlikely scenario where a mistake is caught. Making obvious mistakes such as forgetting arguments likely results in a compiler error anyway, with or without comma, unless that forgotten argument has a default argument or an overload with fewer parameters exists.

§ 3.6.4. Semantic inconsistencies with macros

With the proposed syntax in function calls, `f(0,)` would be valid for both regular functions and for function-style macros. Previously, it would have only been valid if `f` is a function-style macro, but the trailing comma would mean that we are passing two arguments, one of which is empty.

This does result in a bit of surprising behavior for non-variadic cases:



```
void f(int x);
#define F(x)

f(0,);    // OK
F(0,);    // error: two arguments provided, but only one parameter
```

This inconsistency cannot be changed without altering the semantics of the preprocessor, so it is here to stay. However, it also seems largely inconsequential. Users who are uncertain about whether `f` is a macro can choose not to use a trailing comma; it is entirely optional. Furthermore, trailing commas in variadic macros "just work":

```
#define M(...) f(__VA_ARGS__)

// looks like four comma-separated macro arguments (last is empty), but three function arguments
M(0, 1, 2,);
```

§ 3.6.5. Multiple ways to do the same thing

A common and generally good rule of thumb in programming language design is:

” Don't introduce multiple ways to do the same thing.

It could be argued that we can already spell parameter lists without a trailing comma, so it is bad to add a second way. However, this rule is not universally and innately true; it is important to understand why one may follow it:

- Multiple ways of doing the same thing makes a language harder to teach.
- If we can already do the same thing a different way, no value is added to the language.

The first motivation hardly applies because C++ already supports trailing commas in some lists. One could argue the language becomes *more* teachable by fixing this inconsistency and permitting it in all lists. Even if that argument is not accepted, the teaching effort for an optional punctuation character is microscopic.

Secondly, §2. Motivation lays out numerous ways in which trailing commas add value to the language. It would be hard to argue that `// line comments` add no value to the language because `/* block comments */` exist, or that `1s` adds no value because `std::chrono::seconds(1)` exists. It is fine to have multiple spellings of the same feature when there are clear technical benefits to both spellings. There are no hard rules in language design, only trade-offs and rules of thumb.

§ 4. Implementation experience

None.

§ 5. Impact on existing code

The proposal does not alter the meaning of any existing code; it only makes previously ill-formed code valid.

§ 6. Wording

In [\[expr.prim.lambda.capture\]](#), change the grammar as follows:

 `lambda-capture:`
`capture-default .opt`
`capture-list .opt`



In [dcl.pre] paragraph 1, change the grammar as follows:



```
sb-identifier-list:
    sb-identifier .opt
sb-identifier-list, sb-identifier ., sb-identifier-list
```

In [dcl.fct] paragraph 2, change the grammar as follows:



```
parameter-declaration-clause:
    ...
    parameter-declaration-listopt
    parameter-declaration-list , ... .opt
    parameter-declaration-list ...
```

In [dcl.init.general] paragraph 1, change the grammar as follows:



```
[...]
braced-init-list:
    { initializer-list opt }
    { designated-initializer-list opt }
    { }

initializer-list:
    initializer-clause ... opt .opt
initializer-list, initializer-clause ... opt ., initializer-list

designated-initializer-list:
    designated-initializer-clause .opt
designated-initializer-list, designated-initializer-clause ., designated-initializer-list

[...]
```

In [dcl.attr.grammar] paragraph 1, change the grammar as follows:



```
annotation-list:
    annotation ... opt .opt
annotation-list, annotation ... opt ., annotation-list
```

In [temp.pre] paragraph 1, change the grammar as follows:



```
template-parameter-list:
    template-parameter ... opt .opt
template-parameter-list, template-parameter ... opt ., template-parameter-list
```

In [temp.names] paragraph 1, change the grammar as follows:



```
template-argument-list:
    template-argument ... opt .opt
template-argument-list, template-argument ... opt ., template-argument-list
```

If the proposed resolution of [CWGGithub754] at the time of writing has been applied, revert the change as follows:



```
expansion-init-list:  
{ expression-list ⋮opt }  
{ }
```



§ 7. References

- [N5014] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-08-05)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>
- [P0562R2] *Alan Talbot*. Trailing Commas in Base-clauses and Ctor-initializers (2024-04-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0562r2.pdf>
- [P3312R1] *Bengt Gustafsson*. Overload Set Types (2025-04-16)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3312r1.pdf>
- [CWGGithub754] *Jan Schultke*. [stmt.expand] Trailing commas in an expansion-init-list should be permitted (2025-08-24)
<https://github.com/cplusplus/CWG/issues/754>
- [P0562R2ReflectorDiscussion] [isocpp-core] P0562R2 -- how is a compiler intended to find the end of a constructor definition? (2024-06-27)
<https://lists.isocpp.org/core/2024/06/15961.php>
- [CoreReflectorDiscussion] [isocpp-core] P3776R0 Review of grammar changes date = 2025-08-25
<https://lists.isocpp.org/core/2025/08/18487.php>
- [RustTrailingCommas] *Rust Style Guide* — Trailing commas
<https://doc.rust-lang.org/style-guide/#trailing-commas>
- [KotlinTrailingCommas] *What's new in Kotlin 1.4.0* — Trailing comma (2020-08-17)
<https://kotlinlang.org/docs/whatsnew14.html#trailing-comma>
- [ECMAScriptTrailingCommas] *Proposal to allow trailing commas in function parameter lists*
<https://github.com/tc39/proposal-trailing-function-commas>
- [TypeScriptTrailingCommas] *TypeScript 2.0* — Trailing commas in function parameter and argument lists
<https://www.typescriptlang.org/docs/handbook/release-notes/typescript-2-0.html#trailing-commas-in-function-parameter-and-argument-lists>
- [CSharpTrailingCommas] *Proposal: Allow trailing comma in tuples & argument lists*
<https://github.com/dotnet/csharplang/issues/1246>
- [SwiftTrailingCommas] *Allow trailing commas in parameter lists*
<https://forums.swift.org/t/allow-trailing-commas-in-parameter-lists/68549/29>
- [RedditTrailingCommas] *Does your language support trailing commas?*
https://www.reddit.com/r/ProgrammingLanguages/comments/1amsalm/does_your_language_support_trailing_commas/
- [OldNewThing] *Raymond Chen*. On the virtues of the trailing comma (2024-02-09)
<https://devblogs.microsoft.com/oldnewthing/20240209-00/?p=109379>